Shopify Integration

1. Overview

1.1 Description

Shopify Integration connects **Shopify** (e-commerce platform) with **Newo Platform**.

It enables:

- Customer order lookup by email and/or phone number
- Order status retrieval with financial status details
- Automatic customer matching across email and phone identifiers
- OAuth 2.0 authentication with automatic token refresh

This integration is designed for **e-commerce businesses and online retailers** who need **automated order status checking through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a customer asks about their order status → Agent looks up orders by email/phone in Shopify
- When a customer wants to know if their order shipped → Retrieve order with financial status
- When a returning customer calls → Match by phone number and pull all orders
- When a customer provides their email → Search Shopify customers and retrieve order history

1.3 Supported Features

Feature	Supported	Notes
Order Lookup	✅	By customer email and/or phone number
Customer Search	✅	Multi-field search (email OR phone)
Order Financial Status	✅	Returns order ID and financial status per order
OAuth 2.0 Token Management	✅	Auto-refresh with retry (up to 3 attempts)
Dual Connector Isolation	✅	Separate connectors for API and token operations
Order Creation	❌	Read-only order access
Product Catalog	❌	No inventory or product browsing
Order Modification	❌	No order editing or cancellation
Shipping Tracking	❌	Financial status only, no tracking numbers

2. Requirements

Before installation:

- Active **Shopify** store
- **Shopify OAuth App** with Client ID and Client Secret
- Store domain (e.g., `abcde1-11.myshopify.com`)

3. How to Setup

3.1 Step 1 — Authorize in Shopify

1. In Newo platform, set `shopify_shop_id`, `shopify_client_id`, `shopify_client_secret`, and `shopify_redirect_url`
2. Click the Shopify authorization link provided by the platform
3. Log in to your Shopify store admin
4. Authorize data access for the Newo app
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `shopify_oauth_code` attribute

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`shopify_shop_id`	✅	Store domain ID (e.g., `abcde1-11` from `abcde1-11.myshopify.com`)
`shopify_client_id`	✅	OAuth2 Client ID
`shopify_client_secret`	✅	OAuth2 Client Secret
`shopify_redirect_url`	✅	Redirect URL for OAuth callback
`shopify_oauth_code`	✅	One-time OAuth authorization code
`shopify_base_url`	❌	Store API URL (auto-generated from shop_id)
`shopify_timezone`	❌	Store timezone (default: project timezone)

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
   - Exchanges the OAuth code for access and refresh tokens
   - Constructs the base URL from the shop ID
   - Creates HTTP connectors for Shopify API
   - Registers the order lookup tool with the agent

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration uses Shopify **Customers** for contact matching and **Orders** for status retrieval. The agent invokes the order lookup tool when a customer asks about their orders, and results are delivered as an urgent message for the agent to relay.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Shopify

Available Triggers

Trigger	Event	Description
Check Orders	`shopify_check_order_tool`	When the agent needs to look up customer orders
Setup	`project_publish_finish`	Initializes integration on deploy

Available Actions

- **Search Customer** — Find customer by email and/or phone (GET `/admin/api/{version}/customers/search.json`)
- **Get Orders** — Retrieve all orders for a customer (GET `/admin/api/{version}/orders.json`)
- **Exchange Token** — OAuth code for access token (POST `/admin/oauth/access_token`)
- **Refresh Token** — Get new access token via refresh token (POST `/admin/oauth/access_token`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Shopify Integration
    participant T as Shopify OAuth
    participant API as Shopify API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
    I->>T: POST /admin/oauth/access_token (code)
    T-->>I: access_token + refresh_token
    I->>A: Order tool registered

    Note over U,API: Order Lookup
    U->>A: "What's my order status?"
    A->>I: shopify_check_order_tool
    I->>I: Extract email + phone from persona
    I->>API: GET /admin/api/2025-10/customers/search.json?query=(email OR phone)
    API-->>I: Customer data
    alt Customer not found
        I->>A: urgent_message "No customer found"
    else Customer found
        I->>API: GET /admin/api/2025-10/orders.json?customer_id=(id)
        API-->>I: Orders list
        alt Orders found
            I->>A: urgent_message "Orders: (id, status)..."
        else No orders
            I->>A: urgent_message "No orders found"
        end
    end
    A->>U: "Here's your order status..."

    Note over I,API: Token Refresh (on 401)
    I->>API: Any request
```

```
    API-->>I: 401 Unauthorized
    I->>T: POST /admin/oauth/access_token (refresh_token)
    T-->>I: New access_token + refresh_token
    I->>I: Save new tokens
    I->>API: Retry original request
```

CheckOrderFlow

```
flowchart TD
    E["shopify_check_order_tool"] --> EXTRACT["Extract from persona:<br>email,
phone_number<br>(with country code)"]
    EXTRACT --> QUERY{{"Email or phone<br>available?"}}
    QUERY -->|"No"| NOPE["urgent_message:<br>No customer info"]
    QUERY -->|"Yes"| SEARCH["GET /admin/api/2025-10<br>/customers/search.json<br>?
query={email OR phone}"]
    SEARCH --> FOUND{{"Customer<br>found?"}}
    FOUND -->|"No"| NOUSER["urgent_message:<br>No customer found"]
    FOUND -->|"Yes"| ORDERS["GET /admin/api/2025-10<br>/orders.json<br>?customer_id=
{id}<br>&limit=250&status=any"]
    ORDERS --> HAS{{"Orders<br>exist?"}}
    HAS -->|"No"| NOORDERS["urgent_message:<br>No orders found"]
    HAS -->|"Yes"| FORMAT["Format per order:<br>order_id, financial_status"]
    FORMAT --> MSG["urgent_message:<br>'Found orders with statuses:<br>{order
details}'"]
```

SetupFlow

```
flowchart TD
    E["project_publish_finish"] --> CREDS["Extract OAuth credentials:<br>shop_id,
client_id,<br>client_secret, redirect_url"]
    CREDS --> URL["Construct base_url:<br>https://{shop_id}<br>.myshopify.com"]
    URL --> ATTRS["Define 10 project<br>attributes with metadata"]
    ATTRS --> CONN["Create connectors:
<br>shopify_connector<br>shopify_token_connector"]
    CONN --> CODE{{"OAuth code<br>provided?"}}
    CODE -->|"No"| WAIT["Wait for code"]
    CODE -->|"Yes"| TOKEN["POST /admin/oauth<br>/access_token<br>{code, client_id,
<br>client_secret}"]
    TOKEN --> OK{{"Tokens<br>received?"}}
    OK -->|"No"| ERR["Error"]
    OK -->|"Yes"| SAVE["Save access_token<br>and refresh_token<br>Clear oauth_code"]
    SAVE --> TOOL["Register<br>shopify_check_order_tool<br>with agent"]
```

UtilsFlow (Token Refresh)

```
flowchart TD
    REQ["Any API request"] --> HTTP["Send via<br>shopify_connector"]
    HTTP --> STATUS{{"HTTP<br>status?"}}
    STATUS -->|"200"| CHECK{{"Response has<br>error field?"}}
```

```
    CHECK -->|"No"| SUCCESS["shopify_result_success"]
    CHECK -->|"Yes"| ERR["shopify_result_error"]
    STATUS -->|"401"| REFRESH["POST /admin/oauth<br>/access_token<br>{grant_type:
<br>refresh_token}"]
    REFRESH --> TOK_OK{{"New token<br>received?"}}
    TOK_OK -->|"Yes"| RETRY{{"Attempt<br>< 3?"}}
    RETRY -->|"Yes"| SAVE["Save tokens<br>Retry request"]
    RETRY -->|"No"| ERR
    TOK_OK -->|"No"| ERR
    STATUS -->|"4xx/5xx"| ERR
    SAVE --> HTTP
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel). The agent must have access to the customer's email or phone number (collected during conversation or detected from caller ID) to perform order lookups.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify OAuth token exchange completes successfully
2. **Customer Search**: Start a conversation, provide an email or phone — verify Shopify customer is matched
3. **Order Lookup**: Ask "What's my order status?" — verify orders returned with correct financial statuses
4. **No Customer**: Test with unknown email/phone — verify agent responds appropriately
5. **Token Refresh**: Wait for token expiration — verify automatic refresh and retry works

If no action occurs:

- Ensure OAuth credentials are set (`shopify_shop_id` , `shopify_client_id` , `shopify_client_secret`)
- Verify `shopify_access_token` was obtained (check it's not empty after setup)
- Confirm `shopify_oauth_code` was valid and not already used (single-use)
- Check that `shopify_base_url` is correctly formed (`https://{shop_id}.myshopify.com`)
- Verify the customer's email or phone is available in the conversation persona
- Re-publish the project if credentials were changed

> *Note: This integration performs read-only operations in Shopify. Customer and order data is queried but not modified through the agent.*

5. FAQ

Q: Can the integration create or modify orders? A: No. This is a read-only integration for order status lookup. Orders can only be viewed, not created, modified, or cancelled through the agent.

Q: How does the agent know when to look up orders? A: The `shopify_check_order_tool` is registered with the agent and triggers when the agent explicitly states it's checking orders (e.g., "I'm checking your orders now", "Let me look up your order"). The agent's LLM decides when this is appropriate based on the conversation context.

Q: What customer identifiers are used for lookup? A: The integration searches by email and/or phone number. If both are available, it uses an OR query (`email:{email} OR phone:{phone}`). If only one is available, it searches by that field alone.

Q: What order information is returned? A: Each order returns the `order_id` and `financial_status` (e.g., paid, pending, refunded, partially_refunded). The agent uses these details to inform the customer.

Q: What happens if the OAuth token expires? A: The integration automatically detects HTTP 401 responses and refreshes the token using the refresh token. It retries the failed request up to 3 times. A separate HTTP connector is used for token operations to ensure isolation.

Q: Can I connect multiple Shopify stores? A: Each integration instance supports one Shopify store (identified by `shopify_shop_id`). For multiple stores, create separate integration instances.

Q: Which Shopify API version is used? A: The integration uses Shopify REST Admin API version `2025-10` . This is configured internally and does not need to be changed.

Q: Does the integration need specific Shopify app scopes? A: Yes. The OAuth app must have at minimum `read_customers` and `read_orders` scopes to perform customer search and order retrieval.